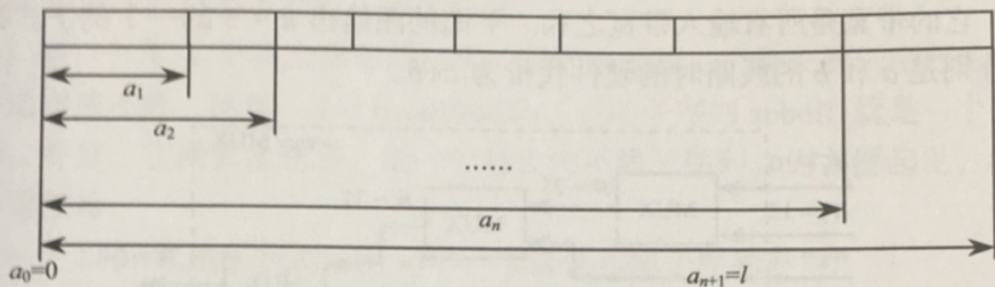


04动态规划

代码题

Q1切长钢管

5、假设我们要将一根大型长钢管锯成若干段。我们在要锯的地方打上标记，一共是 n 个标记。这些标记距离钢管左端的距离，从左到右，分别是 $a_1, a_2, \dots, a_n$ 厘米。这个钢管的总长是 l 厘米， $l > A[n]$ （参见下图）。当我们将钢管锯为两截时，需要的代价与当时被锯钢管的长度成正比，比例是每一厘米 p 元。



- (a) 请用动态规划的方法设计一个算法来找出最优的顺序来完成这 n 处的切割，使总的代价最小。分析你的算法的复杂度。（提示：用 $a_0 = 0$ 和 $a_{n+1} = l$ 表示钢管左端和右端位置。用 $[a_i, a_j]$ 表示从标记 a_i 到 标记 a_j 这段钢管。用 $C(i, j)$ 表示完成对 $[a_i, a_j]$ 这段钢管的切割任务所需的最小代价，也就是完成所有 a_k ($i < k < j$) 处的切割所需的代价。找出 $C(i, j)$ 的归纳公式。）
- (b) 以下面数据为例，用你在 (a) 中的算法找出最优的切割顺序和总代价。请显示你的计算过程。
- $a_1 = 2, a_2 = 5, a_3 = 9, a_4 = 14, l = 15, p = 1$ 。

基本思想

这道题是经典的最优二叉搜索树或矩阵链乘类型的区间动态规划问题。

核心逻辑：

1. 定义状态：设 $C(i, j)$ 为切割区间 $[a_i, a_j]$ 内所有标记点 $a_{i+1}, \dots, a_{j-1}$ 所需的最小代价。
 - 我们的目标是求 $C(0, n+1)$ ，即从头 $a_0 = 0$ 到尾 $a_{n+1} = l$ 之间切完 $a_1 \dots a_n$ 的最小代价。
2. 边界条件：
 - 当 $j = i + 1$ 时，区间 (a_i, a_j) 内部没有切割点（因为没有 k 满足 $i < k < j$ ），所以不需要切，代价为 0。
 - $C(i, i + 1) = 0$ 。
3. 状态转移方程：
 - 对于区间 $[a_i, a_j]$ ，如果要在中间切一刀，必须选一个切割点 a_k （其中 $i < k < j$ ）。
 - 切这一刀的直接代价是当前这一段钢管的长度： $p \times (a_j - a_i)$ 。
 - 切完后，问题被分解为两部分：左边 $[a_i, a_k]$ 和右边 $[a_k, a_j]$ 。
 - 我们需要遍历所有可能的 k ，选出使总代价最小的那个。
 - 递推公式：

$$C(i, j) = (a_j - a_i) \cdot p + \min_{i < k < j} \{C(i, k) + C(k, j)\}$$

复杂度分析：

- 状态数： i 和 j 都在 0 到 $n + 1$ 之间，所以大约有 $O(n^2)$ 个状态。
- 计算每个状态：需要遍历 k （从 $i + 1$ 到 $j - 1$ ），耗时 $O(n)$ 。
- 总复杂度： $O(n^3)$ 。

代码块

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <cstring>
5
6  using namespace std;
7
8  const int MAXN = 105;
9  int dp[MAXN][MAXN]; // DP表
10 int points[MAXN];    // 存储所有坐标点: 0, a1, ..., an, L
11
12 /**
```

```

13  * @param p 单位切割代价
14  * @param n 切割点数量 (不含头尾)
15  * @param L 钢管总长
16  * @param a 切割点坐标数组 (下标1到n)
17  */
18  int cutSteelBars(int p, int n, int L, int a[]) {
19      // 1. 构造完整的坐标点数组 points
20      // points[0] = 0
21      // points[1...n] = a[1...n]
22      // points[n+1] = L
23      points[0] = 0;
24      for (int i = 1; i <= n; ++i) {
25          points[i] = a[i];
26      }
27      points[n + 1] = L;
28
29      int m = n + 2; // 总点数 (0 到 n+1)
30
31      // 2. 初始化 DP 表
32      // 初始化为无穷大
33      memset(dp, 0x3f, sizeof(dp));
34
35      // 相邻两个点之间无法再切 (span = 1), 代价为 0
36      // 例如 dp[0][1], dp[1][2]...
37      for (int i = 0; i < m - 1; ++i) {
38          dp[i][i + 1] = 0;
39      }
40
41      // 3. 区间 DP
42      // len 表示区间跨度 (j - i), 从 2 开始 (中间至少有一个切点)
43      for (int len = 2; len < m; ++len) {
44          // 枚举起点 i
45          for (int i = 0; i + len < m; ++i) {
46              int j = i + len;
47
48              // 当前这一段的物理长度 (切这一刀的固定代价)
49              int current_cost = (points[j] - points[i]) * p;
50
51              // 枚举分割点 k (i < k < j)
52              // k 对应的是 points 数组的下标, 代表第 k 个切点
53              for (int k = i + 1; k < j; ++k) {
54                  int cost = dp[i][k] + dp[k][j] + current_cost;
55                  if (cost < dp[i][j]) {
56                      dp[i][j] = cost;
57                  }
58              }
59      }

```

```

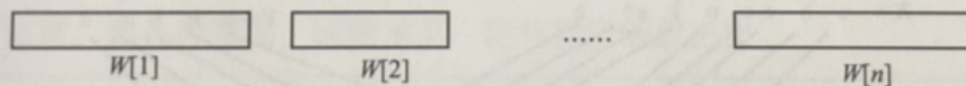
60     }
61
62     // 返回从点 0 到 点 n+1 的最小代价
63     return dp[0][n + 1];
64 }
65
66 int main() {
67     // 测试数据 (题目 (b) 问)
68     // a1=2, a2=5, a3=9, a4=14, l=15, p=1
69     int n = 4;
70     int L = 15;
71     int p = 1;
72     int a[MAXN];
73     a[1] = 2; a[2] = 5; a[3] = 9; a[4] = 14;
74
75     cout << "Min Cost: " << cutSteelBars(p, n, L, a) << endl;
76     // 预期输出: 35
77
78     return 0;
79 }
80

```

$i \setminus j$	0	1	2	3	4	5
0	-	0	5	14	28	35
1	-	-	0	7	19	26
2	-	-	-	0	9	16
3	-	-	-	-	0	6
4	-	-	-	-	-	0
5	-	-	-	-	-	-

Q2焊接多根钢管

6. 如下图所示，顺序放好的 n 根钢管的重量分别为 $W[i]$ ($1 \leq i \leq n$)。



我们需要把它们依照顺序焊成一根钢管，但每次焊接可任意选两根相邻的钢管来焊接。每次焊接的代价与被焊两段钢管的总重量成正比。为简单起见，把代价定为被焊两段钢管的总重量。例如， $W[1]=5$, $W[2]=1$, $W[3]=2$ ，如果先把 $W[1]$ 和 $W[2]$ 焊好，代价为 $5+1=6$ ，焊好的这块有重量 6。再把 $W[3]$ 焊上，又要代价 $6+2=8$ ，总代价是 14。但如果先焊 $W[2]$ 和 $W[3]$ ，再焊 $W[1]$ ，则总代价为 11。

(a) 用动态规划的方法设计一个算法计算出最优的焊接顺序使总代价最小。

(b) 应用你上面的算法求出以下 5 根钢管的最优焊接顺序和总代价： $W[1]=8$, $W[2]=1$, $W[3]=7$, $W[4]=2$, $W[5]=9$ 。

基本思想

这道题其实就是经典的石子合并 (Matrix Chain Multiplication 变体) 问题，属于区间动态规划。

核心逻辑：

1. 定义状态：

设 $DP[i][j]$ 表示将第 i 根到第 j 根钢管 (区间 $W[i \dots j]$) 完全焊接在一起所需的最小总代价。

2. 边界条件：

- 当 $i = j$ 时，只有一根钢管，不需要焊接，代价为 0。
- 即 $DP[i][i] = 0$ 。

3. 状态转移方程：

- 要把 $W[i \dots j]$ 这一大段合并成一根，最后一步一定是把两部分焊在一起。
- 设最后一步的分界点是 k ($i \leq k < j$)，即把 $[i, k]$ 和 $[k+1, j]$ 这两段焊起来。
- 这一次焊接的直接代价是： $\text{sum}(i, j) = \sum_{x=i}^j W[x]$ (即这段钢管的总重)。
- 总代价 = 左边子问题的最小代价 + 右边子问题的最小代价 + 当前合并代价。
- 递推公式：

$$DP[i][j] = \min_{i \leq k < j} \{DP[i][k] + DP[k+1][j]\} + \sum_{x=i}^j W[x]$$

复杂度：

- 状态数 $O(n^2)$ ，每个状态需要遍历 k ，复杂度 $O(n^3)$ 。

表格 $DP[i][j]$ 表示合并第 i 根到第 j 根钢管的最小代价。最终结果在右上角 $DP[2][1] = 62$ 。

$i \setminus j$	1	2	3	4	5
1	0	9	24	36	62
2	-	0	8	18	37
3	-	-	0	9	27
4	-	-	-	0	11
5	-	-	-	-	0

代码块

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <climits>
5
6  using namespace std;
7
8  const int MAXN = 105;
9  int W[MAXN];          // 钢管重量
10 int S[MAXN];           // 前缀和，用于快速算区间总重
11 int dp[MAXN][MAXN];   // DP表
12
13 // 获取区间 [i, j] 的总重量
14 // 公式: S[j] - S[i-1]
15 int get_sum(int i, int j) {
16     return S[j] - S[i - 1];
17 }
18
19 int main() {
20     int n;
21     if (!(cin >> n)) return 0; // 输入钢管数量
22
23     // 1. 输入 n 根钢管的重量
24     for (int i = 1; i <= n; ++i) {
25         cin >> W[i];
26     }
27
28     // 2. 计算前缀和 S
29     S[0] = 0;
30     for (int i = 1; i <= n; ++i) {
31         S[i] = S[i - 1] + W[i];
```



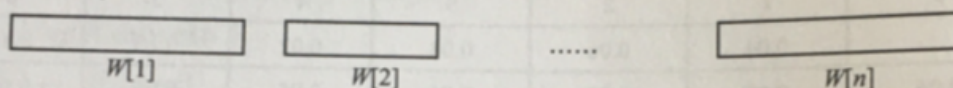
```

32     }
33
34     // 3. 初始化 DP
35     // 单根钢管不需要焊接，代价为 0
36     for (int i = 1; i <= n; ++i) {
37         dp[i][i] = 0;
38     }
39
40     // 4. 区间 DP 主循环
41     // len 从 2 开始（至少合并 2 根）
42     for (int len = 2; len <= n; ++len) {
43         for (int i = 1; i <= n - len + 1; ++i) {
44             int j = i + len - 1;
45             dp[i][j] = INT_MAX; // 初始化为无穷大
46
47             // 本次合并的直接代价（这是 Problem 6 的核心修改点）
48             // 无论 k 在哪里切，最后一步合并的代价都是这段钢管的总重
49             int cost_merge = get_sum(i, j);
50
51             // 枚举分割点 k (i <= k < j)
52             // 将 [i, k] 和 [k+1, j] 合并
53             for (int k = i; k < j; ++k) {
54                 // 总代价 = 左边累积 + 右边累积 + 本次合并代价
55                 int total = dp[i][k] + dp[k + 1][j] + cost_merge;
56
57                 if (total < dp[i][j]) {
58                     dp[i][j] = total;
59                 }
60             }
61         }
62     }
63
64     // 5. 输出结果
65     cout << dp[1][n] << endl;
66
67     return 0;
68 }
69

```

Q3焊接多根钢管（不同的递归函数

我们需要把它们依照顺序焊成一根钢管，但每次焊接可任意选两根相邻的钢管来焊接。每次焊接的代价与被焊两段钢管中的较重的一根的重量成正比。为简单起见，把代价定为被焊两段钢管中的较重的一根的重量。例如， $W[1] = 5, W[2] = 1, W[3] = 2$ ，如果先把 $W[1]$ 和 $W[2]$ 焊好，代价为 5，再把 $W[3]$ 焊上，又要代价 6，总代价是 11。但如果先焊 $W[2]$ 和 $W[3]$ ，再焊 $W[1]$ ，则总代价为 $2+5=7$ 。



(a) 用动态规划的方法设计一个算法计算出最优的焊接顺序使总代价最小。

(b) 应用你上面的算法求出以下 5 根钢管的最优焊接顺序和总代价： $W[1] = 8, W[2] = 1, W[3] = 7, W[4] = 2, W[5] = 9$ 。

(a) 动态规划设计

1. 状态定义

我们依然需要定义 $DP[i][j]$ 为合并区间 i 到 j 的最小代价。

但是，这次的合并代价依赖于子区间的总重量。

我们需要维护两个信息：

- $S[i][j]$: 区间 $W[i \dots j]$ 的总重量 (这个是固定的, $\sum_{k=i}^j W[k]$)。
- $DP[i][j]$: 将区间 $W[i \dots j]$ 合并成一根的最小累积代价。

2. 递归方程

最后一步合并发生在分界点 k 处，将 $[i, k]$ 和 $[k+1, j]$ 合并。

这次的合并代价是 $\max(\text{weight}(i, k), \text{weight}(k+1, j))$ 。

即 $\max(S[i][k], S[k+1][j])$ 。

状态转移方程：

$$DP[i][j] = \min_{i \leq k < j} \{DP[i][k] + DP[k+1][j] + \max(S[i][k], S[k+1][j])\}$$

- $S[i][k]$ 是左段的总重。
- $S[k+1][j]$ 是右段的总重。

3. 边界条件

- $DP[i][i] = 0$ (单根无需合并)。

代码块

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <limits>
```



```

5
6 using namespace std;
7
8 const int MAXN = 105;
9 int W[MAXN];          // 钢管重量
10 int S[MAXN];          // 前缀和, 用于快速算区间总重
11 int dp[MAXN][MAXN];  // DP表
12
13 // 获取区间 [i, j] 的总重量
14 int get_sum(int i, int j) {
15     return S[j] - S[i - 1];
16 }
17
18 int main() {
19     int n;
20     if (!(cin >> n)) return 0; // 输入钢管数量
21
22     // 1. 输入 n 根钢管的重量
23     for (int i = 1; i <= n; ++i) {
24         cin >> W[i];
25     }
26
27     // 2. 计算前缀和 S
28     S[0] = 0;
29     for (int i = 1; i <= n; ++i) {
30         S[i] = S[i - 1] + W[i];
31     }
32
33     // 3. 初始化 DP
34     // 单根钢管不需要焊接, 代价为 0
35     for (int i = 1; i <= n; ++i) {
36         dp[i][i] = 0;
37     }
38
39     // 4. 区间 DP 主循环
40     // len 从 2 开始 (至少合并 2 根)
41     for (int len = 2; len <= n; ++len) {
42         for (int i = 1; i <= n - len + 1; ++i) {
43             int j = i + len - 1;
44             dp[i][j] = INT_MAX; // 初始化为无穷大
45
46             // 枚举分割点 k (i <= k < j)
47             // 将 [i, k] 和 [k+1, j] 合并
48             for (int k = i; k < j; ++k) {
49                 // 左半部分总重
50                 int weight_left = get_sum(i, k);
51                 // 右半部分总重

```

```

52         int weight_right = get_sum(k + 1, j);
53
54         // 本次合并的直接代价
55         int cost_merge = max(weight_left, weight_right);
56
57         // 总代价 = 左边累积 + 右边累积 + 本次合并
58         int total = dp[i][k] + dp[k + 1][j] + cost_merge;
59
60         if (total < dp[i][j]) {
61             dp[i][j] = total;
62         }
63     }
64 }
65 }
66
67 // 5. 输出结果
68 cout << dp[1][n] << endl;
69
70 return 0;
71 }
72

```

最终结果 $DP[1][5] = 40$ 。

$i \setminus j$	1	2	3	4	5
1	0	8	15	24	40
2	-	0	7	15	25
3	-	-	0	7	16
4	-	-	-	0	9
5	-	-	-	-	0